pfi

Python for Fixed Income

COURSE 4 · MACHINE LEARNING

Confusion Matrix, Precision, and Recall

WEEK 2 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

A probability becomes a flag

Day 6
a model probability of default in this file

Today
a threshold turns it into a flag, 1 or 0

Then
count the flags against what happened

Real data, one place and year
30,000 card accounts, Taiwan, 2005. UCI, CC BY 4.0. Credit there differs from US credit.

Never model features
SEX, EDUCATION, MARRIAGE, AGE. Every week 2 model reads the other 19 columns.



Default rate 22.12 percent in every part. Today: training and validation rows only.

What the output is called

Allowed names

model probability of default in this file

flagged at threshold t

the model's score

Never called

a credit decision

an approval, a decline

a lending policy

a risk grade for a real person

The file describes accounts at one lender, in one country, in one year. A model fitted to it describes that file.

The all-zero model and its accuracy

A model that flags nobody

0.7788

accuracy on the validation rows

0 of 1,327

defaults flagged

Day 6's model, flagged at 0.5

0.8123

accuracy on the validation rows

324 of 1,327

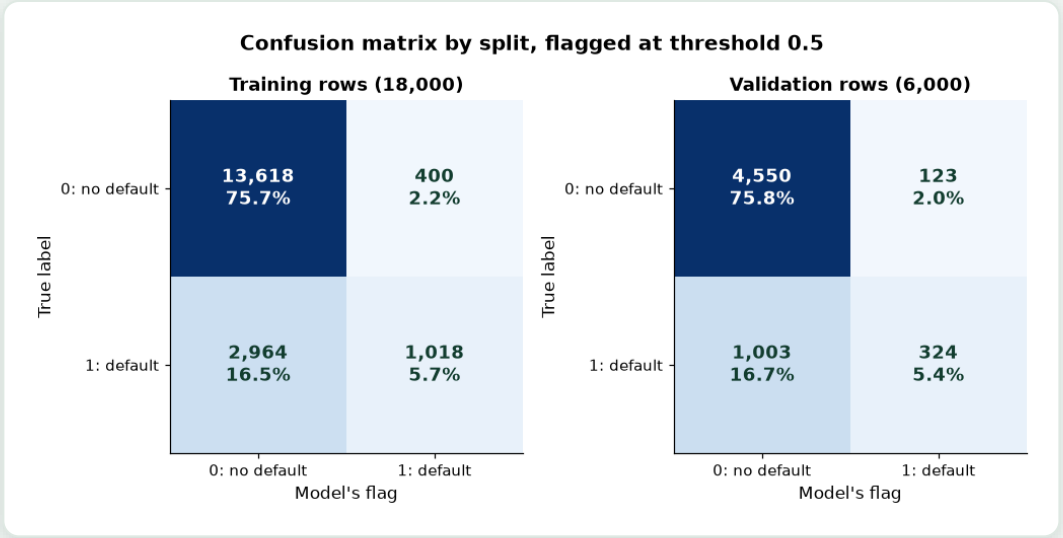
defaults flagged

The all-zero model is right on every account that paid, 4,673 of 6,000, because they are most of the rows. On a file where most rows are 0, accuracy mostly measures how many rows are 0.

Accuracy is always shown beside the all-zero model's, and never used to choose.

The confusion matrix, cell by cell

	Flagged 0	Flagged 1
Actual 0: paid	**TN** 4,550	**FP** 123
Actual 1: default	**FN** 1,003	**TP** 324



Rows: 4,673 paid, 1,327 defaulted. Columns: 447 flagged, 5,553 not.

FN = 1,003: the defaults accuracy hid.

Precision, recall, and F1, for the defaults

Precision

`TP / (TP + FP)`

Of the accounts flagged, the share that defaulted

0.7248

Recall

`TP / (TP + FN)`

Of the defaults, the share flagged

0.2442

F1

`2TP / (2TP + FP + FN)`

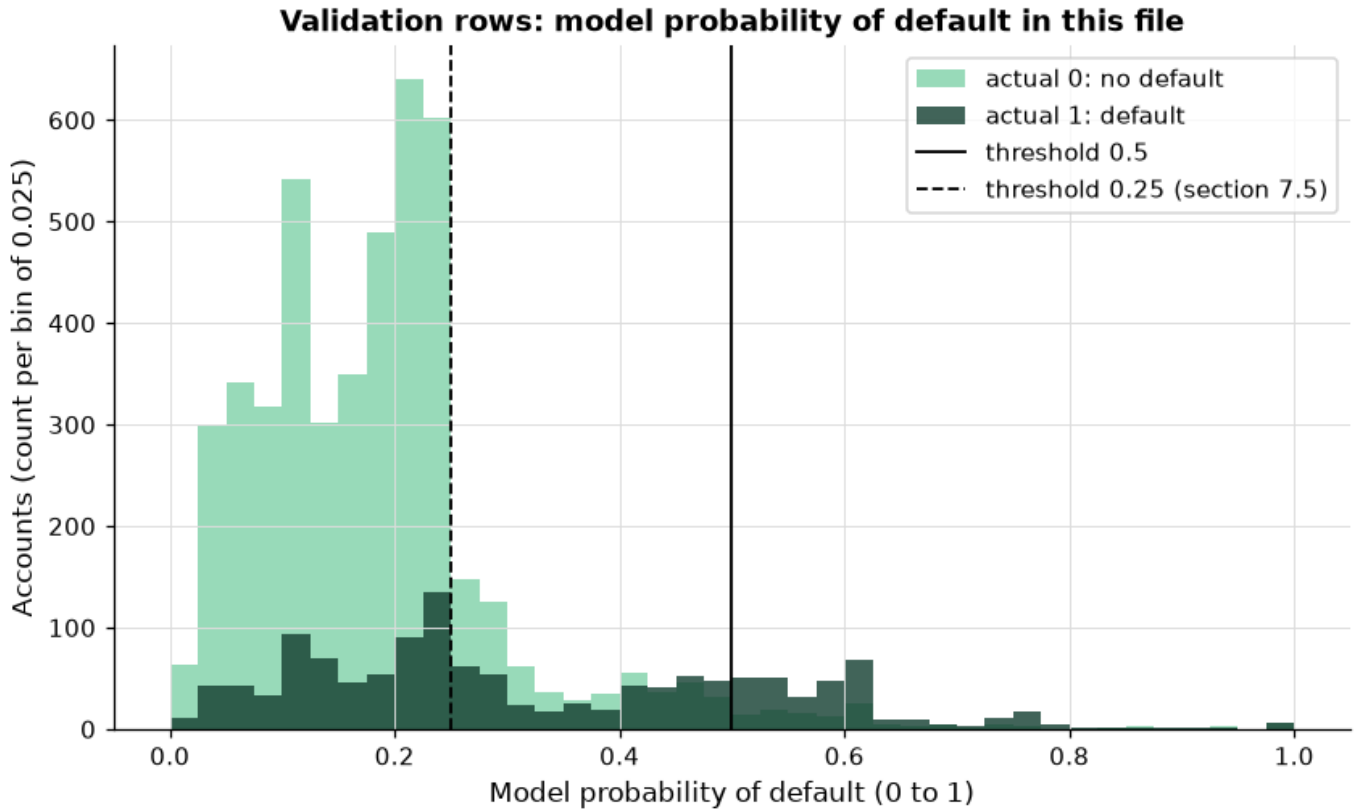
Harmonic mean: high only when both are

0.3653

Precision reads down the flagged 1 column. Recall reads along the actual 1 row. Neither uses the true negatives, so neither is flattered by a file full of zeros.

At 0.5: a flag is a default about 7 times in 10, and about 3 defaults in 4 go unflagged. 31 tests pass.

Threshold 0.5 and what it catches



447

accounts flagged, 7.45% of the rows

324 of 1,327

defaults flagged (recall 0.2442)

predict against proba >= t

predict flags above 0.5; the course flags at least t. Here: the same flags.

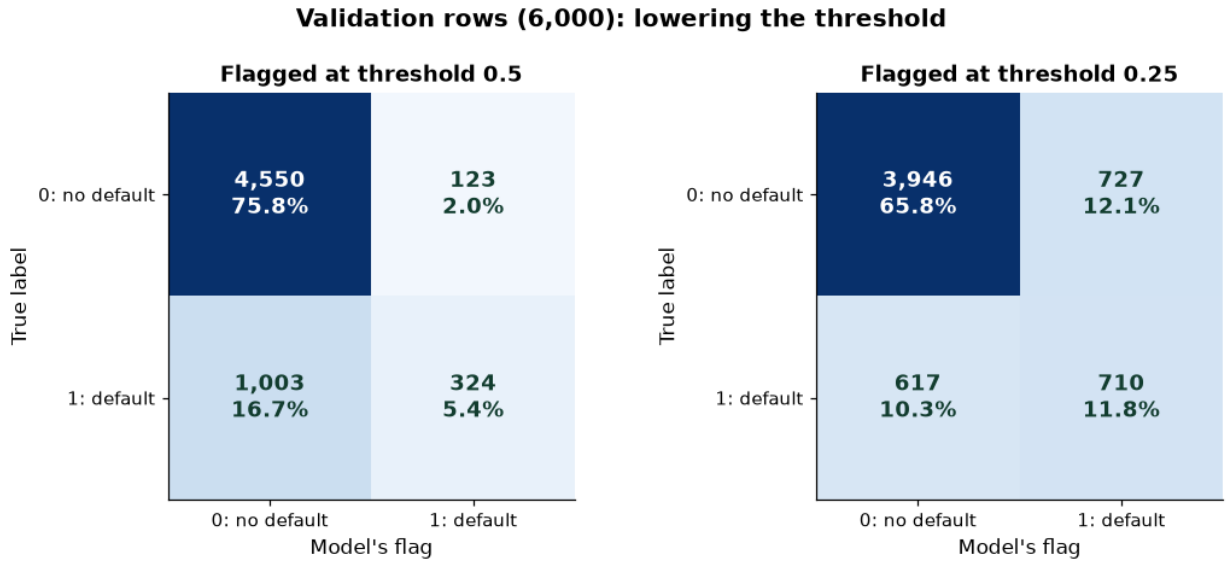
What each error costs, as arithmetic on counts

False positive

An account that paid, flagged: attention spent on it.

False negative

A default, not flagged: the flag did not point to it.



1.56

false flags per extra default caught

From 0.5 to 0.25: recall 0.2442 to 0.5350, precision 0.7248 to 0.4941; 386 more defaults caught, 604 more false flags. Counting a miss as w false flags, the smaller total is at 0.5 for w = 1 and at 0.25 for w = 2.

The weight is a choice the file cannot make. No count here is a rule a lender uses.

classification_report, read by row

```
              precision    recall  f1-score   support
           0     0.8194    0.9737    0.8899      4673
           1     0.7248    0.2442    0.3653      1327
    accuracy                         0.8123      6000
   macro avg     0.7721    0.6089    0.6276      6000
weighted avg     0.7985    0.8123    0.7739      6000
```

Row 1	the defaults: the numbers of 7.3
Row 0	0 as positive: recall 0.9737 is not the defaults'
macro avg	plain average of the two rows
weighted avg	leans toward class 0, the larger

The course reads row 1. A summary quoting recall 0.9737 for this model read the wrong row.

Excel side by side: four COUNTIFS

Excel side by side (tested in Excel)

`F5 =COUNTIFS($A$2:$A$6001,1,$B$2:$B$6001,">="&F$1)`	TP 324
`F4 =COUNTIFS($A$2:$A$6001,1,$B$2:$B$6001,"<"&F$1)`	FN 1,003
`F3 =COUNTIFS($A$2:$A$6001,0,$B$2:$B$6001,">="&F$1)`	FP 123
`F2 =COUNTIFS($A$2:$A$6001,0,$B$2:$B$6001,"<"&F$1)`	TN 4,550
`=F5/(F5+F3)  and  =F5/(F5+F4)`	precision 0.7248, recall 0.2442
`=2*F5/(2*F5+F3+F4)`	F1 0.3653
`=COUNTIF($A$2:$A$6001,0)/COUNT($A$2:$A$6001)`	all-zero accuracy 0.7788

Sheet valid: A the outcome, B the model's probability, threshold in F1. Typing 0.25 there recounts everything: TP 710, FP 727.

Counts equal Python's exactly; every ratio within 1e-12.

Git: a change of threshold in the log

1. Code

mlkit.py and test_mlkit.py

2. Log at 0.5

results.csv, 4 decimals

3. Log at 0.25

the same model, threshold moved

```
!git -C "{work}" diff --word-diff-regex="[^,[:space:]]+" results.csv
logistic_19_columns_scaled,validation,,,[-0.5000,0.8123,0.7248,0.2442,0.3653-]
    {+0.2500,0.7760,0.4941,0.5350,0.5137+}
```

Columns: model, rows, roc_auc, average_precision, threshold, accuracy, precision, recall, f1. ROC AUC and average precision stay empty until day 8. Only the threshold and the four metrics are marked.

Plain --word-diff splits at spaces; a CSV row has none, so tell Git a word ends at a comma.

The AI check

The docstring is the prompt. The code that comes back runs, and has one modelling bug.

1

Docstring

Which rows fit, which rows report, and the units: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: recall computed with pos_label=0: 0.9737, the accounts that paid, against 0.2442 for the defaults.

Code that runs is not the same as code that is right.

What you can do now

- Flagging nobody scores 0.7788: accuracy hides the rare class
- Four counts: TN, FP, FN, TP
- Precision down the flagged column, recall along the default row
- A lower threshold trades false flags for caught defaults
- In classification_report, the defaults are row 1

Week 2, Day 3 ROC curves and choosing a threshold on validation rows